

计算机程序设计基础(2)

--- C++程序设计(3)

孙甲松

sunjiasong@tsinghua.edu.cn

电子工程系 信息认知与智能系统研究所

罗姆楼6-104

电话: 62796193/13901216180

2020. 3.

3.4 静态成员

- ◆ 同一个类所定义的对象之间是相互独立的，若想在多个对象之间共享数据、传递信息，则可以定义类的静态数据成员。
- ◆ 同一个类无论定义多少个对象，静态成员在整个程序中只有一个，同一个类的不同的对象访问的静态成员是同一个，从而可以在不同对象之间传递信息。同时外界不可见。
- ◆ 若一个成员函数只存取静态成员，则这个成员函数应该声明为类的静态成员函数。静态成员函数在整个程序中也只有一个。

例如：

3.4 静态成员(续1)

```
class Myclass {  
    public:  
        Myclass(int i);  
        ~Myclass( );  
        void Print( );  
    private:  
        int member;  
        static int global;  
};
```

则无论用Myclass类定义多少个Myclass对象，静态数据成员global在整个程序中只有一个。Myclass类的不同对象访问的global是同一个。

3.4 静态成员(续2)



一个实例:

```
#include <iostream>
using namespace std;
class Point {
public:
    Point(int x=0, int y=0);
    Point(Point &p);
    ~Point( )
    { Print( ); count--; }
    void Print( );
    static void PrintCount( ) // 静态成员函数
    { cout << "ID=" << count << ' '; }
private:
    int x, y;
    static int count; // 静态成员
};
```

```

Point::Point(int x, int y) : x(x), y(y)
{
    count++;    Print( );
}
Point::Point(Point &p) : x(p.x), y(p.y)
{
    count++;    Print( );
}
void Point::Print( )
{
    PrintCount( );
    cout << '(' << x << ', ' << y << ')' << endl;
}
int Point::count=0; //静态成员初始化，必须在主程序和类外
void main( )
{
    Point a(1,2), b(3,4), c(a);
    a.Print( );
    b.Print( );
    c.Print( );
    a.PrintCount( );
    Point::PrintCount( ); // 静态成员函数不依附于任何对象
}

```

运行结果:

ID=1 (1,2)

ID=2 (3,4)

ID=3 (1,2)

ID=3 (1,2)

ID=3 (3,4)

ID=3 (1,2)

ID=3 ID=3 ID=3 (1,2)

ID=2 (3,4)

ID=1 (1,2)

请按任意键继续...

注意：虽然静态数据成员count是类的私有成员，但赋初值只能在主函数外，类外单独进行：

```
int Point::count=0;
```

但必须使用类名限定Point::。否则 int count=0; 就变成定义外部变量并赋初值0。

如果在类中定义静态变量同时赋初值：

```
static int count=0;
```

编译结果为：

error C2864: “Point::count”: 只有静态常量整型数据成员才可以在类中初始化

编译系统提示：只有静态常量整型数据成员才可以在类中初始化，也就是说，只有常静态成员：

```
static int const count=0; 或 static const int count=0;
```

才能在类中赋初值。

3.4.1 常静态成员

```
#include <iostream>
using namespace std;
class Point {
public:
    Point(int x=0, int y=0) : x(x), y(y) { }
    void Print( )
    { PrintCount( ); cout <<'('<<x <<','<<y<<')'<<endl;}
    static void PrintCount( ) // 静态成员函数
    { cout <<"ID=" << count << ' '; }
private:
    int x, y;
    static const int count=999; // 常静态成员在类内赋初值
};

void main( )
{ Point a(1,2), b(3,4);
  a.Print( ); b.Print( );
}
```



3.5 友元

- ◆ C++提供了友元机制，放松对于类中私有成员访问的严格限制，让不是本类成员函数的其它函数也能直接访问本类的私有成员，这会方便编写程序，提高执行效率。
- ◆ 方法是：白名单制，在类内逐个列出其友元函数，每个函数说明前面加上关键字 `friend`。
- ◆ 每一个友元都必须在此类中声明，这有利于此类对访问权限的控制。

例如：

```

#include <iostream>
using namespace std;
class TRS
{ public:
    TRS(long n=0, long d=0);
    void Print( );
    friend TRS Add(TRS &r1, TRS &r2);
    // 注意: Add不是TRS类的成员函数, 这里将
    // Add函数说明为TRS类的友元
private:
    long N, D;
};
TRS Add(TRS &r1, TRS &r2)//注意: Add前面没有TRS::
{ TRS r(0,0); //这里对象r必须赋初值,因为TRS无缺省值
  r.N = r1.N * r2.D + r1.D * r2.N;
  r.D = r1.D * r2.D;
  return r;
}

```

3.5 友元(续1)

```
TRS::TRS(long n, long d)
{ N=n; D=d; }
void TRS::Print()
{ cout<<"N=" <<N<<" D="<<D<<endl; }
void main()
{ TRS a(1,3), b(2,9), c;
  c = Add(a, b);
  c.Print();
}
```

运行结果:
N=15 D=27
请按任意键继续...

Add函数不是TRS类的成员函数，并且需要同时访问一个以上TRS类的对象的私有成员，所以把Add函数定义为TRS类的友元。

3.5 友元(续2)

- ◆ 一个类可以定义为另一个类的友元

```
class A {  
    friend class B;  
    .....  
}
```

- ◆ 这样类B的所有成员函数都成为类A的友元函数，可以直接访问类A的私有成员。
- ◆ 实际上在VS编译器上，不能单独把类B的一个或几个成员函数定义成另一个类A的友元，而只能把整个类B定义成另一个类A的友元，让类B的所有成员函数都成为类A的友元函数。

3.6 嵌套类

函数外定义类名全局可见，为了限制类名的可见范围、作用域及其访问权限，可以将一个类声明在另一个类之中，形成嵌套类。

```
class enclose {  
    public:  
        class inner {  
            public : void g(int p);  
            private: int y;  
        };  
        void f(int q);  
    private: int x;  
}
```

3.6 嵌套类(续1)

类inner被称为类enclose的嵌套类，受类enclose的作用域限制，另一个类中也可以嵌套一个同名的inner类，也可以直接定义一个同名的inner类，这些inner类相互之间既无影响也无任何关联。

现在要想使用inner类，必须加前缀**enclose::**

例如，在类外定义inner的成员函数g()，必须写成：

```
void enclose::inner::g(int p)
{ y = p; }
```

而想用inner类定义对象，必须写成：

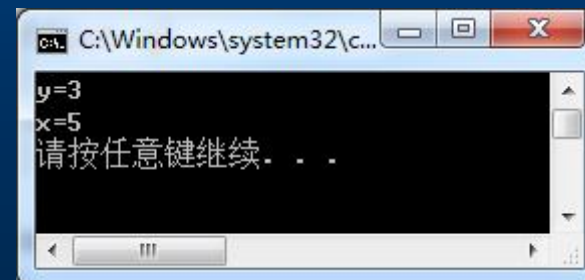
```
enclose::inner a;
```

类的嵌套仅仅是语法上的嵌套，嵌套类的成员与包围它的类无关，反之亦然。

```

#include <iostream>
using namespace std;
class enclose {
public:
    class inner {
        public : void g(int p);
                void print() { cout << "y=" << y << endl; }
        private: int y;
    };
    void f(int q);
    void print() { cout << "x=" << x << endl; }
private: int x;
};
void enclose::f(int q)
{ x = q; }
void enclose::inner::g(int p)
{ y = p; }
void main( )
{ enclose::inner a;
  enclose b;
  a.g(3);  a.print( );
  b.f(5);  b.print( );
}

```



3.7 类的向前引用

由于种种原因，类的使用出现的类的说明之前，需要首先声明一下类的符号，这就是类的向前引用说明，与函数的向前引用说明类似（但有些编译器不需要）

```
class B; // 类的向前引用
```

```
class A {  
    public: void f(B b);  
    .....
```

```
};
```

```
class B {  
    public: void g(A a);  
    .....
```

```
};
```

- 这样做的目的是，告诉编译器f函数参数中出现的类型B不是未定义符号，而是一个类名，这样编译器就不会把B按未定义符号报错。
- f中定义参数B b是形参，并不需要像实际定义类的对象那样真正开辟内存，因此向前引用说明就够了。
- 不能用向前引用说明的符号B在类A中定义对象，也不能在类A的内联成员函数中使用该类对象。
- 但可以用向前引用说明的符号B在类A中定义对象指针。因为所有指针都是4字节（32位编译系统上）。
- 如果上面的程序改为：

```
class B; // 类的向前引用
class A {
public: B b;
    .....
};
class B {
public: A a;
    .....
};
```

- 在类A中以类B对象b作为其成员，同时在类B中以类A对象a作为其成员，这样是无法通过编译的，因为B b;是要真正定义一个B类对象b，需要知道B类的真正定义形式和大小。
- 同时这样写也是不允许的，因为类A和类B是相互无限递归嵌套定义的。

3.8 this指针

- 一种自引用机制，使成员函数可以访问到对象本身，是隐含的指针参数，this指针指向该成员函数正在操作的对象。
- this指针只能在成员函数内使用。
- 类的静态成员函数中无this指针。
- this指针不能被更新。
(因为this的类型为 `X * const`，常指针)

例如：

```
#include <iostream>
using namespace std;
class MyClass {
    public:  MyClass(int x, int y);
           void Print( );
    private: int x, y;
};
MyClass::MyClass(int x, int y) //构造函数形参与成员同名
{ this->x=x; this->y=y;} // MyClass对象的this指针。此处
                        // 必须用this指针，否则出现：
                        // x=x; y=y; 赋值无意义
void MyClass::Print( )
{ cout << x <<" "<< this->y << endl; }
```

```

class YourClass {
    public: YourClass(int i, int j);
           void Print( );
    private: MyClass my;
};
YourClass::YourClass(int i, int j) : my(i, j){ }
void YourClass::Print( )
{  this->my.Print( ); } // YourClass对象的this指针,此处
                        // this指针可以省略
void main( )    // YourClass完全克隆MyClass
{  YourClass you(5, 10);
   you.Print( );
}

```

运行结果:

5 10

请按任意键继续. . .

3.9 类的组合

- 类的成员数据是另一个类的对象。
- 从而在已有的抽象的基础上实现更复杂的抽象。
- `class B {`
.....
`};`
`class A {`
 `public: .....`
 `private: B b1, b2, b3;`
`};`

例如：一个三角形类的成员是三个点类的对象。

```
class Point {  
    public: Point(int x, int y);  
           void Display( ) const;  
    private: int X, Y;  
};  
  
class Triangle {  
    public: Triangle (int x1, int y1, int x2, int y2,  
                    int x3, int y3, int w);  
           void Display( ) const;  
    private: Point p1, p2, p3;  
           int W;  
};
```

3.9 类的组合(2)

- Triangle类的构造函数不仅要对本类的成员变量赋初值，还要构造各个对象成员并赋初值。

```
Point::Point(int x, int y)
```

```
{ X=x; Y=y; cout << "Point " << X << ' ' << Y << endl; }
```

```
void Point::Display() const // 常成员函数,函数中只读  
{ cout << X << ' ' << Y << endl; }
```

```
Triangle::Triangle (int x1, int y1, int x2, int y2,  
    int x3, int y3, int w) : p1(x1,y1), p2(x2,y2), p3(x3,y3)  
{ W=w; cout << "Triangle\n"; } // p1,p2,p3的构造顺序任意  
void Triangle::Display() const // 常成员函数,函数中只读  
{ p1.Display(); p2.Display(); p3.Display();  
  cout<<W<<endl;  
}
```

3.9 类的组合(3)

```
void main( )  
{ Triangle t(1,2,3,4,5,6,7);  
  t.Display( );  
}
```

- ✓ 组合类的构造函数的调用顺序是：先顺序调用执行内嵌对象的构造函数(构造顺序由定义时的对象顺序所确定)，再执行本类的构造函数。
- ✓ 析构函数的执行顺序与构造顺序正相反。
- ✓ 这里先执行3次Point构造函数构造p1,p2,p3内嵌对象，再执行Triangle构造函数内语句。

运行结果是：

Point 1,2

Point 3,4

Point 5,6

Triangle

1,2

3,4

5,6

7

请按任意键继续...

又例如：线段（Line）类，一条线由两个点组成。

```
#include <iostream>
#include <cmath>
using namespace std;
class Point      //Point类的声明
{
public:
    Point(int xx=0, int yy=0)
    { X=xx; Y=yy;
      cout<<"Point构造函数被调用 "<<X<<","<<Y<<endl;
    }
    Point(Point &p);
    ~Point() {cout<<"Point析构函数被调用 "<<X<<","<<Y<<endl;}
    int GetX() { return X; }
    int GetY() { return Y; }
private:
    int X,Y;
};
Point::Point(Point &p) //复制构造函数的实现
{ X=p.X; Y=p.Y;
  cout<<"Point复制构造函数被调用 "<<X<<","<<Y<<endl;
}
```

//类的组合

class Line //Line类的声明

{

public: //外部接口

Line (Point xp1, Point xp2);

Line (Line &);

~Line (){ cout<<"Line析构函数被调用"<<endl; }

double GetLen(){ return len; }

private: //私有数据成员

Point p1, p2; //Point类的对象p1,p2

double len; // 线段的长度

};

//组合类的构造函数

Line:: Line (Point xp1, Point xp2) : p1(xp1), p2(xp2)

{ cout<<"Line构造函数被调用"<<endl;

double x=(double)(p1.GetX()-p2.GetX());

double y=(double)(p1.GetY()-p2.GetY());

len=sqrt(x*x+y*y);

}

//组合类的复制构造函数

```
Line:: Line (Line &l) : p1(l.p1), p2(l.p2)
```

```
{
```

```
    cout<<"Line复制构造函数被调用"<<endl;
```

```
    len=l.len;
```

```
}
```

```
void main( ) //主函数
```

```
{
```

```
    Point myp1(1,1),myp2(4,5); //建立2个Point类的对象
```

```
    Line line(myp1,myp2);      //用两个点建立Line类的对象
```

```
    Line line2(line);         //利用复制构造函数建立一个新对象
```

```
    cout<<"The length of the line is:";
```

```
    cout<<line.GetLen( )<<endl;
```

```
    cout<<"The length of the line2 is:";
```

```
    cout<<line2.GetLen( )<<endl;
```

```
}
```

运行结果:

Point构造函数被调用 1,1
Point构造函数被调用 4,5
Point复制构造函数被调用 4,5
Point复制构造函数被调用 1,1
Point复制构造函数被调用 1,1
Point复制构造函数被调用 4,5
Line构造函数被调用
Point析构函数被调用 1,1
Point析构函数被调用 4,5
Point复制构造函数被调用 1,1
Point复制构造函数被调用 4,5
Line复制构造函数被调用
The length of the line is:5
The length of the line2 is:5
Line析构函数被调用
Point析构函数被调用 4,5
Point析构函数被调用 1,1
Line析构函数被调用
Point析构函数被调用 4,5
Point析构函数被调用 1,1
Point析构函数被调用 4,5
Point析构函数被调用 1,1
请按任意键继续. . .

若把Line的构造函数改写为:

```
Line:: Line (Point xp1, Point xp2)
{ p1=xp1;
  p2=xp2;
  cout<<"Line构造函数被调用"<<endl;
  double x=(double)(p1.GetX( )-p2.GetX( ));
  double y=(double)(p1.GetY( )-p2.GetY( ));
  len=sqrt(x*x+y*y);
}
```

若把Line的复制构造函数改写为:

```
Line:: Line (Line &l)
{ p1=l.p1;
  p2=l.p2;
  cout<<"Line复制构造函数被调用"<<endl;
  len=l.len;
}
```

运行结果:

```
Point构造函数被调用 1,1
Point构造函数被调用 4,5
Point复制构造函数被调用 4,5
Point复制构造函数被调用 1,1
Point构造函数被调用 0,0
Point构造函数被调用 0,0
Line构造函数被调用
Point析构函数被调用 1,1
Point析构函数被调用 4,5
Point构造函数被调用 0,0
Point构造函数被调用 0,0
Line复制构造函数被调用
The length of the line is:5
The length of the line2 is:5
Line析构函数被调用
Point析构函数被调用 4,5
Point析构函数被调用 1,1
Line析构函数被调用
Point析构函数被调用 4,5
Point析构函数被调用 1,1
Point析构函数被调用 4,5
Point析构函数被调用 1,1
请按任意键继续...
```

修改前运行结果:

```
Point构造函数被调用 1,1
Point构造函数被调用 4,5
Point复制构造函数被调用 4,5
Point复制构造函数被调用 1,1
Point复制构造函数被调用 1,1
Point复制构造函数被调用 4,5
Line构造函数被调用
Point析构函数被调用 1,1
Point析构函数被调用 4,5
Point复制构造函数被调用 1,1
Point复制构造函数被调用 4,5
Line复制构造函数被调用
The length of the line is:5
The length of the line2 is:5
Line析构函数被调用
Point析构函数被调用 4,5
Point析构函数被调用 1,1
Line析构函数被调用
Point析构函数被调用 4,5
Point析构函数被调用 1,1
Point析构函数被调用 4,5
Point析构函数被调用 1,1
请按任意键继续...
```

通过对比会发现：

虽然在Line构造函数 `Line::Line (Point xp1, Point xp2)` 后面没有加：`p1(xp1), p2(xp2)`，但此Line构造函数在执行时，还是会首先调用缺省值的Point构造函数构造Point对象p1、p2，并分别赋初值为(0,0)，然后再执行 `p1=xp1; p2=xp2;` 把形参对象xp1、xp2的值赋给p1、p2。

而：

`Line::Line (Point xp1, Point xp2) : p1(xp1), p2(xp2)` 的写法，Line构造函数执行时，直接调用Point的复制构造函数构造Point对象p1、p2，并分别赋初值为xp1、xp2。

相比起来，`Line::Line (Point xp1, Point xp2)` 的写法多了两次对象赋值，效率更低一些。

同样，虽然在Line复制构造函数 `Line::Line (Line &l)` 后面没有加 `: p1(l.p1), p2(l.p2)`，但Line复制构造函数在执行时，会首先调用缺省值的Point构造函数构造Point对象p1、p2，并分别赋初值为(0,0)，然后再执行 `p1=l.p1; p2=l.p2;` 把形参对象l的两个内嵌对象p1、p2的值赋给p1、p2。

而：

`Line::Line (Line &l) : p1(l.p1), p2(l.p2)` 的写法，Line复制构造函数执行时，直接调用Point的复制构造函数构造Point对象p1、p2, 并分别赋初值为 `l.p1`、`l.p2`。

所以比较起来，`Line::Line (Line &l)`的写法多了两次对象赋值，效率更低一些。

3.10 指向类的静态成员的指针

```
#include <iostream>
using namespace std;
class Point      //Point类声明
{
public:           //外部接口
    Point(int xx=0, int yy=0);    //构造函数
    Point(Point &p);              //复制构造函数
    int GetX() { return X; }
    int GetY() { return Y; }
    static int countP;           //静态数据成员引用性说明，这是公有的
private:                       //私有数据成员
    int X,Y;
};
Point::Point(int xx, int yy)
{  X=xx; Y=yy;  countP++;
}
Point::Point(Point &p)
{  X=p.X; Y=p.Y;
  countP++; // 这是缺省复制构造函数不可能自动实现的
}
```

3.10 指向类的静态成员的指针(2)

```
int Point::countP=0; //静态数据成员赋初值
void main() //主函数实现
{
    int *count=&Point::countP; //声明一个int型指针，指向类的静态成员
    Point A(4,5); //声明对象A
    cout<<"Point A,"<<A.GetX()<<","<<A.GetY();
    cout<<" Object ID="<<*count<<endl; //直接通过指针访问公有静态数据成员
    Point B(A); //声明对象B
    cout<<"Point B,"<<B.GetX()<<","<<B.GetY();
    cout<<" Object ID="<<*count<<endl; //直接通过指针访问公有静态数据成员
    Point C(1,2); //声明对象C
    cout<<"Point C,"<<C.GetX()<<","<<C.GetY();
    cout<<" Object ID="<<C.countP<<endl; //直接访问公有静态数据成员
    Point D(C); //声明对象D
    cout<<"Point D,"<<D.GetX()<<","<<D.GetY();
    cout<<" Object ID="<<D.countP<<endl; //直接访问公有静态数据成员
}
```

3.10 指向类的静态成员的指针(3)

运行结果:

Point A,4,5 Object ID=1

Point B,4,5 Object ID=2

Point C,1,2 Object ID=3

Point D,1,2 Object ID=4

3.11 指向静态成员函数的函数指针

```
#include <iostream>
using namespace std;
class Point    //Point类声明
{
public: //外部接口
    Point(int xx=0, int yy=0) {X=xx; Y=yy; countP++;} //构造函数
    Point(Point &p);    //复制构造函数
    int GetX( ) {return X;}
    int GetY( ) {return Y;}
    static void GetC( ) {cout<<" Object ID="<<countP<<endl;}
                        //静态成员函数
private:    //私有数据成员
    int X,Y;
    static int countP;    //静态数据成员, 注意是私有的
};
Point::Point(Point &p)
{ X=p.X;    Y=p.Y;    countP++; }
```

3.11 指向静态成员函数的函数指针

```
int Point::countP=0; //静态数据成员定义性说明，初始化，使用类名限定
void main( ) //主函数实现
{ void (*gc)( )=Point::GetC;
    //声明一个指向函数的指针，指向类的静态成员函数
    Point A(4,5); //声明对象A
    cout<<"Point A,"<<A.GetX( )<<","<<A.GetY( );
    gc( ); //输出对象ID，直接通过指针调用静态数据成员函数
    Point B(A); //声明对象B
    cout<<"Point B,"<<B.GetX( )<<","<<B.GetY( );
    gc( ); //输出对象ID，直接通过指针调用静态数据成员函数
    Point C(7,8); //声明对象C
    cout<<"Point C,"<<C.GetX( )<<","<<C.GetY( );
    Point::GetC( ); //输出对象ID，通过静态成员函数输出静态数据成员
}
```

3.11 指向静态成员函数的函数指针

运行结果:

Point A,4,5 Object ID=1

Point B,4,5 Object ID=2

Point C,7,8 Object ID=3

请按任意键继续...

3.12 类与对象数组

用已声明的类，可以一个一个定义对象，也可以一次定义多个对象：对象数组

例如：

```
#include <iostream>
using namespace std;
class Point    //Point类声明
{
public: //外部接口
    Point(int xx=0, int yy=0)
    { X=xx; Y=yy;
      cout <<"Construt:("<<X<<","<<Y<<")\n"; } //构造函数
    Point(Point &p) { X=p.X; Y=p.Y;} //复制构造函数
    ~Point() {cout <<"Destrut:("<<X<<","<<Y<<")\n"; } //析构函数
    int GetX() { return X; }
    int GetY() { return Y; }
private: //私有数据成员
    int X,Y;
};
```

```
void main( ) //主函数
{ Point A[5]={Point(1,2), Point(3,4), Point(5,6), Point(7,8), Point(9,10) };
    // 注意一下，类对象数组的赋初值形式，用无名对象序列赋初值
    int i;
    for (i=0; i<5; i++)
        cout<<"Point A["<<i<<"]="<<A[i].GetX()<<","<<A[i].GetY()<<")\n";
}
```

注意：用无名对象序列的构造替代了对象数组的构造，不再需要逐个构造对象数组的元素。

运行结果：

Construt:(1,2)
Construt:(3,4)
Construt:(5,6)
Construt:(7,8)
Construt:(9,10)
Point A[0]=(1,2)
Point A[1]=(3,4)
Point A[2]=(5,6)
Point A[3]=(7,8)
Point A[4]=(9,10)
Destrut:(9,10)
Destrut:(7,8)
Destrut:(5,6)
Destrut:(3,4)
Destrut:(1,2)
请按任意键继续...

注意：其中的构造顺序和析构顺序完全相反

3.13 类与动态对象

用已声明的类，除了可以定义对象和对象数组，也可以用new创建动态对象或动态对象数组。例如：

```
#include <iostream>
using namespace std;
class Point    //Point类声明
{
public: //外部接口
    Point(int xx=0, int yy=0)
    {   X=xx; Y=yy;
        cout <<"Construt:("<<X<<","<<Y<<")\n"; } //构造函数
    Point(Point &p) {   X=p.X; Y=p.Y;}    //复制构造函数
    ~Point( ) {cout<<"Destrut:("<<X<<","<<Y<<")\n"; } //析构函数
    int GetX( ) {return X;}
    int GetY( ) {return Y;}
private:    //私有数据成员
    int X,Y;
};
```

```

void main( ) //主函数
{ Point *A, *B, *C;
  int i;
  A = new Point(1,2); // 注意：动态类对象的赋初值形式
  B = new Point[5]; // 创建了5个动态类对象,形成动态类对象
  for (i=0; i<5; i++) // 数组，会触发5次构造函数
    B[i]=Point(i+1, i+2); // 无名对象构造后一旦赋值完立刻析构掉
  cout<<"Point A"<<"=("<< A->GetX( )<<","<<A->GetY( )<<")\n";
  for (i=0; i<5; i++)
    cout<<"Point B["<<i<<"]="<< B[i].GetX( )<<","<<B[i].GetY( )<<")\n";
  C = B+4;
  for (; C>=B; C--)
    cout<<"Point=("<< C->GetX( )<<","<<C->GetY( )<<")\n";
  delete A; // delete指针A所指对象，会触发一次析构函数
  delete [ ]B; // delete指针B所指对象数组，会触发5次析构函数
} // 如果不delete掉动态对象, 运行结束时也不会自动触发析构函数
运行结果:

```

Construt:(1,2)
Construt:(0,0)
Construt:(0,0)
Construt:(0,0)
Construt:(0,0)
Construt:(0,0)
Construt:(1,2)
Destrut:(1,2)
Construt:(2,3)
Destrut:(2,3)
Construt:(3,4)
Destrut:(3,4)
Construt:(4,5)
Destrut:(4,5)
Construt:(5,6)
Destrut:(5,6)
Point A=(1,2)

Point B[0]=(1,2)
Point B[1]=(2,3)
Point B[2]=(3,4)
Point B[3]=(4,5)
Point B[4]=(5,6)
Point=(5,6)
Point=(4,5)
Point=(3,4)
Point=(2,3)
Point=(1,2)
Destrut:(1,2)
Destrut:(5,6)
Destrut:(4,5)
Destrut:(3,4)
Destrut:(2,3)
Destrut:(1,2)
请按任意键继续...

又例如:

```
#include <iostream>
#include <cstring>
using namespace std;
class Strings {
public:
    Strings(const char *s); // 构造函数
    Strings(Strings &p);    // 复制构造函数
    ~Strings();             // 析构函数
    void Print();
    void Set(const char *s);
private:
    int length;
    char *str;
};
Strings::Strings(Strings &p)
{
    length = strlen(p.str);
    str = new char[length+1]; //为对象中str 申请动态内存块
    strcpy(str, p.str);
    cout << "复制构造函数被调用！" << str<< endl;
}
```

```

Strings::Strings(const char *s)
{
    length = strlen(s);
    str = new char[length+1]; //为对象中str 申请动态内存块
    strcpy(str, s);
    cout << "构造函数被调用！" << str<< endl;
}

Strings::~Strings( )
{
    cout << "析构函数被调用！" << str << endl;
    delete [ ]str; // 释放对象中str 所指的动态内存块
}

void Strings::Print( )
{
    cout << str << endl;
    cout << "length=" << length << endl;
}

void Strings::Set(const char *s)
{
    length = strlen(s);
    delete [ ]str; //先释放掉对象中str 所指的旧的动态内存块
    str = new char[length+1]; //为对象中str 申请新的动态内存块
    strcpy(str, s);
}

```

```
void main( )  
{ Strings *S1;  
  S1 = new Strings("Hello!");  
  Strings S2(*S1);  
  S1->Print( );  
  S2.Print( );  
  S1->Set("New String!");  
  S1->Print( );  
  S2.Print( );  
  delete S1;  
}
```

注意： 其中S1对象指针所指的是动态对象， S2是静态对象

运行结果是：

构造函数被调用！ Hello!
复制构造函数被调用！ Hello!
Hello!
length=6
Hello!
length=6
New String!
length=11
Hello!
length=6
析构函数被调用！ New String!
析构函数被调用！ Hello!
请按任意键继续...

注意： 其中的new创建一个动态对象触发了Strings的构造函数，调用new申请字符串动态内存。其中的delete释放动态对象触发了Strings的析构函数，调用delete释放字符串动态内存。

附：一个综合了成员函数、友元函数，函数参数传递对象，函数返回对象的例子。

注意：其中各种对象的构造和析构的时间点与先后顺序，也注意：成员函数与友元函数书写与使用上的差别。

```
#include <iostream>
using namespace std;
class complex {    //复数类声明
public:
    complex(double r=0, double i=0) : r(r), i(i) // 构造函数
    { cout << "Constructor! "; display( ); }
    complex(complex &c) : r(c.r), i(c.i) // 复制构造函数
    { cout << "Copy Constructor!"; display(); }
    ~complex( ) { cout << "Destructor! "; display( ); } // 析构函数
    complex add(complex c2); //成员函数
    friend complex sub(complex c1, complex c2); //友元函数
    void display(); //输出复数
private:    //私有数据成员
    double r; //复数实部
    double i; //复数虚部
};
```

```

complex complex::add(complex c2) //成员函数实现
{ return complex(r+c2.r, i+c2.i); } //创建临时无名对象作为返回值
complex sub(complex c1, complex c2) //友元函数
{ complex t;
  t.r = c1.r-c2.r;  t.i = c1.i-c2.i;
  return t;
} //此函数也可以简化为: return complex(c1.r-c2.r, c2.i-c2.i);
void complex::display( )
{ cout<<"("<<r<<","<<i<<)"<<endl; }
void main( ) //主函数
{ complex c1(5,4),c2(2,10),c3;          //声明复数类的对象
  cout<<"c1="; c1.display( );
  cout<<"c2="; c2.display( );
  c3=sub(c1,c2);
  cout<<"c3=c1-c2=";
  c3.display( );
  c3=c1.add(c2);
  cout<<"c3=c1+c2=";
  c3.display( );
}

```

Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Copy Constructor!(2,10)
Copy Constructor!(5,4)
Constructor! (0,0)
Copy Constructor!(3,-6)
Destructor! (3,-6)
Destructor! (5,4)
Destructor! (2,10)
Destructor! (3,-6)
c3=c1-c2=(3,-6)
Copy Constructor!(2,10)
Constructor! (7,14)
Destructor! (2,10)
Destructor! (7,14)
c3=c1+c2=(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (5,4)
请按任意键继续...

构造对象c1, 初值(5,4)
构造对象c2, 初值(2,10)
构造对象c3, 缺省初值(0,0)
打印c1值(5,4)
打印c2值(2,10)
自右向左传递函数参数, 复制构造c2
自右向左传递函数参数, 复制构造c1
构造局部对象t, 缺省初值(0,0)
由对象t复制构造一个无名对象返回
析构局部对象t
析构参数c1
析构参数c2
赋值完成后, 析构sub返回的无名对象
打印c3值(3,-6)
为add传递函数参数, 复制构造c2
构造一个无名对象返回
析构参数c2
赋值完成后, 析构add返回的无名对象
打印c3值(7,14)
析构对象c3
析构对象c2
析构对象c1

把上例中成员函数、友元函数的函数参数都改为引用方式。注意其中各种对象的构造和析构的时间点与顺序：

```
#include <iostream>
using namespace std;
class complex {    //复数类声明
public:
    complex(double r=0, double i=0) : r(r), i(i) // 构造函数
    { cout << "Constructor! "; display( ); }
    complex(complex &c) : r(c.r), i(c.i) // 复制构造函数
    { cout << "Copy Constructor!"; display(); }
    ~complex( ) { cout << "Destructor! "; display( ); } //析构函数
    complex add(complex &c2); //成员函数
    friend complex sub(complex &c1, complex &c2); //友元函数
    void display(); //输出复数
private:    //私有数据成员
    double r; //复数实部
    double i; //复数虚部
};
```

```

complex complex::add(complex &c2)           //成员函数
{ return complex(r+c2.r, i+c2.i); } //创建临时无名对象作为返回值
complex sub(complex &c1, complex &c2) //友元函数
{ complex t;
  t.r = c1.r-c2.r;  t.i = c1.i-c2.i;
  return t;
}
void complex::display( )
{ cout<<"("<<r<<","<<i<<)"<<endl; }
void main( ) //主函数
{ complex c1(5,4),c2(2,10),c3;           //声明复数类的对象
  cout<<"c1="; c1.display( );
  cout<<"c2="; c2.display( );
  c3=sub(c1,c2);
  cout<<"c3=c1-c2=";
  c3.display( );
  c3=c1.add(c2);
  cout<<"c3=c1+c2=";
  c3.display( );
}

```

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Constructor! (0,0)
Copy Constructor!(3,-6)
Destructor! (3,-6)
Destructor! (3,-6)
c3=c1-c2=(3,-6)
Constructor! (7,14)
Destructor! (7,14)
c3=c1+c2=(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (5,4)
请按任意键继续...
```

引用方式使得传递参数只需要传递对象的地址，不需要复制构造对象，相应也就不需要析构传递参数所构造的临时对象，这可以提高程序的执行效率。

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Copy Constructor!(2,10)
Copy Constructor!(5,4)
Constructor! (0,0)
Copy Constructor!(3,-6)
Destructor! (3,-6)
Destructor! (5,4)
Destructor! (2,10)
Destructor! (3,-6)
c3=c1-c2=(3,-6)
Copy Constructor!(2,10)
Constructor! (7,14)
Destructor! (2,10)
Destructor! (7,14)
c3=c1+c2=(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (5,4)
请按任意键继续...
```

第4次作业:

见网络学堂第4次作业

2题必做, 1题选做